Module Review: Generative Models and Post-Training of Neural Networks

1 Introduction to Generative Modeling

1.1 The Goal of Generative Models

Generative models aim to learn the underlying distribution of a training dataset to generate new, similar data samples *(Lecture 24)*. This can be framed as sampling from an unknown data distribution, where the model learns a parameterized approximation of this distribution. There are two primary settings:

1. **Unconditional Generation**: The model takes a source of randomness (e.g., a vector sampled from a Gaussian distribution) and produces a new, unconstrained example *(Lecture 24)*.

$$\text{Randomness} \rightarrow G \rightarrow \text{New Example}$$

2. **Conditional Generation**: The model takes both randomness and a conditioning input (e.g., a text prompt, class label) to produce an example consistent with the condition *(Lecture 24)*. This is the more common and practical setting for applications like text-to-image models and large language models (LLMs).

$$(\text{Conditioning}, \text{Randomness}) \rightarrow G \rightarrow \text{New Example}$$

1.2 Why Simple Approaches Fail

To appreciate modern generative models, it is useful to understand why more intuitive approaches are insufficient *(Lecture 24)*.

cites the corresponding lecture

1.2.1 Gradient Ascent on a Classifier

A naive idea is to use a pre-trained classifier. To generate a "cat" image, one could start with random noise and perform gradient ascent on the classifier's "cat" score *(Lecture 24)*.

This fails due to the **Manifold Hypothesis**, which posits that natural data (like images of cats) lies on a thin, low-dimensional manifold within the high-dimensional space of all possible pixel values *(Lecture 24)*. Starting from random noise, which is far off-manifold, gradient ascent does not guide the image onto the manifold. Instead, it finds an adversarial example—an image that looks like noise to a human but maximally excites the classifier's internal features for "cat" *(Lecture 24)*.

1.2.2 Standard Autoencoders

An autoencoder consists of an encoder E that maps data x to a low-dimensional latent code z, and a decoder D that reconstructs $\hat{x}$ from z *(Lecture 24)*. A generative approach might be to sample a random z and feed it to the decoder.

This fails because the decoder is only trained on the specific, structured submanifold of latent codes produced by the encoder for the training data. Sampling a random z from a simple distribution like a Gaussian will almost certainly result in a point that is off this submanifold. The decoder, having never seen such a code, will produce a poor or nonsensical output. This is a problem of extrapolation *(Lecture 24)*.

2 Variational Autoencoders (VAEs)

VAEs are a type of generative model that extends the autoencoder framework to create a structured latent space suitable for generation *(Lecture 24)*.

2.1 Core Idea and Architecture

The goal of a VAE is to make the decoder robust to variations in the latent code z *(Lecture 24)*. This is achieved by introducing randomness into the encoding process and regularizing the structure of the latent space.

The VAE architecture modifies the standard autoencoder:

1. The **encoder** E takes an input x and outputs the parameters of a probability distribution over the latent space, typically a Gaussian with mean $\mu_Q(x)$ and covariance $\Sigma_Q(x)$. This distribution is denoted $Q(z|x)$ *(Lecture 24)*.

2. A latent vector z is **sampled** from this distribution: $z \sim Q(z|x) = \mathcal{N}(\mu_Q(x), \Sigma_Q(x))$ *(Lecture 24)*.

3. The **decoder** D takes the sampled z and reconstructs the output $\hat{x}$ *(Lecture 24)*.

This process is illustrated in a problem from discussion *(Discussion 13, Problem 2a)*. The problem asks to draw a block diagram of a VAE, which would show an input x passing through a "Probabilistic Encoder" to produce parameters μ and σ, from which a latent vector z is sampled (using a source of noise ϵ). This z is then passed to a "Probabilistic Decoder" to produce the reconstruction $\hat{x}$.

2.2 The VAE Loss Function

The VAE is trained by minimizing a combined loss function that balances reconstruction quality with latent space regularity *(Lecture 24)*.

$$L_{total} = L_{reconstruction} + \beta \cdot L_{distributional}$$

1. **Reconstruction Loss**: Measures the difference between the original input x and the reconstruction $\hat{x}$, e.g., using Mean Squared Error: $\|x - \hat{x}\|^2$ *(Lecture 24)*.

2. **Distributional Loss**: A regularizer that forces the distribution $Q(z|x)$ produced by the encoder to be close to a chosen prior distribution $P(z)$, from which we will sample during generation. A standard choice for the prior is $\mathcal{N}(0, I)$. This loss is typically measured by the Kullback-Leibler (KL) divergence, $D_{\mathrm{KL}}(Q(z|x)\|P(z))$ *(Lecture 24)*.

The overall loss function is often written as the negative Evidence Lower Bound (ELBO) *(Discussion 13, Problem 2b)*.

$$L_{VAE}(x) = \mathbb{E}_{z\sim Q(z|x)}[\log p(x|z)] - D_{\mathrm{KL}}(Q(z|x)\|P(z))$$

Here, the first term is the expected log-likelihood of the reconstruction (reconstruction loss), and the second is the KL divergence (regularization loss).

2.2.1 Kullback-Leibler (KL) Divergence

The KL divergence measures the dissimilarity between two probability distributions, Q and P *(Lecture 24)*. For discrete distributions, it is defined as:

$$D_{\mathrm{KL}}(P\|Q) = \sum_x P(x)\log\frac{P(x)}{Q(x)}$$

It is asymmetric, meaning $D_{\mathrm{KL}}(P\|Q) \neq D_{\mathrm{KL}}(Q\|P)$ in general *(Lecture 24, Discussion 12, Problem 1a)*. In VAEs, we use $D_{\mathrm{KL}}(Q(z|x)\|P(z))$, which penalizes the encoder for producing latent codes that are unlikely under our desired prior $P(z)$ *(Lecture 24)*.

The KL divergence can be expressed in terms of entropy $H(P)$ and cross-entropy $H(P,Q)$ *(Discussion 12, Problem 1b)*.

$$D_{\mathrm{KL}}(P\|Q) = H(P,Q) - H(P) = \left(-\sum_x P(x)\log Q(x)\right) - \left(-\sum_x P(x)\log P(x)\right)$$

Practice Problem: KL Divergence *(Discussion 12, Problem 1a)* Let $p(x) = \begin{cases} 0.5 & \text{when } x = 1 \\ 0.5 & \text{when } x = -1 \end{cases}$ and $q(x) = \begin{cases} 0.1 & \text{when } x = 1 \\ 0.9 & \text{when } x = -1 \end{cases}$. Show that $D_{\mathrm{KL}}(p\|q) \neq D_{\mathrm{KL}}(q\|p)$.

Solution:

$$D_{\mathrm{KL}}(p\|q) = 0.5\times\log\frac{0.5}{0.1} + 0.5\times\log\frac{0.5}{0.9} \approx 0.74$$
$$D_{\mathrm{KL}}(q\|p) = 0.1\times\log\frac{0.1}{0.5} + 0.9\times\log\frac{0.9}{0.5} \approx 0.53$$

Since $0.74 \neq 0.53$, the KL divergence is not symmetric.

Incorporates specific problems in discussion

For a Gaussian encoder distribution $Q(z|x) = \mathcal{N}(\mu_Q, \Sigma_Q)$ and a standard normal prior $P(z) = \mathcal{N}(0, I_k)$, the KL divergence has a convenient closed-form solution that can be used directly in the loss function *(Lecture 24)*:

$$D_{\mathrm{KL}}(\mathcal{N}(\mu_Q,\Sigma_Q)\|\mathcal{N}(0,I_k)) = \frac{1}{2}\left(\mathrm{Tr}(\Sigma_Q) + \mu_Q^T\mu_Q - k - \log\det(\Sigma_Q)\right)$$

2.3 The Reparameterization Trick

To train a VAE with SGD, we need to backpropagate through the sampling step $z \sim Q(z|x)$, which is a stochastic operation. The reparameterization trick makes this possible by isolating the randomness *(Lecture 24)*.

Instead of sampling z directly, we first sample a random vector $\vec{v}$ from a fixed standard normal distribution $\mathcal{N}(0, I_k)$. Then, z is computed as a deterministic function of the encoder outputs and this random noise *(Lecture 24)*:

$$\vec{z} = \vec{\mu}_Q + \Sigma_Q^{1/2}\vec{v}, \quad \text{where } \vec{v} \sim \mathcal{N}(0, I_k)$$

In this formulation, gradients can flow back through the deterministic computations to update the encoder's parameters (μ_Q, Σ_Q), while the random noise $\vec{v}$ is treated as a non-differentiable input to the model *(Lecture 24)*.

2.4 The Information Bottleneck and Generation

The two loss terms in a VAE create an **information bottleneck** *(Lecture 24, Discussion 13, Problem 2c)*. The reconstruction loss encourages the encoder to pack as much information about x into z as possible. In contrast, the KL divergence loss pushes z's distribution toward the simple, uninformative prior, which discards information about the specific input x. This tension forces the model to learn a compressed representation that captures only the most essential features *(Discussion 13, Problem 2d)*.

At generation time, the encoder is discarded. We sample a latent vector z directly from the prior distribution, $z \sim P(z) = \mathcal{N}(0, I)$, and pass it through the trained decoder D to generate a new data sample $\hat{x}$ *(Lecture 24, Discussion 13, Problem 2e)*.

3 Diffusion Models

Diffusion models are a class of generative models that have achieved state-of-the-art results, particularly in image generation. They operate by learning to reverse a gradual noising process *(Lecture 26)*.

3.1 Core Idea

The core idea is to define a two-stage process *(Lectures 26, 27)*:

1. **Forward (Diffusion) Process**: This is a fixed, non-learned process that gradually adds noise to a data sample x_0 over a sequence of time steps $t = 1, \ldots, T$. This creates a Markov chain of increasingly noisy samples $x_1, x_2, \ldots, x_T$. If T is large enough, x_T becomes indistinguishable from pure noise (e.g., $\mathcal{N}(0, I)$) *(Lecture 26)*.

$$x_0 \to x_1 \to x_2 \to \cdots \to x_T$$

2. **Reverse (Generative) Process**: The model learns to reverse this process. Starting from pure noise x_T, it iteratively denoises the sample step-by-step to produce a clean sample $\hat{x}_0$ *(Lectures 26, 27)*.

$$\hat{x}_0 \leftarrow \hat{x}_1 \leftarrow \cdots \leftarrow x_T$$

3.2 The Forward Process

The forward process is defined by a sequence of conditional probabilities, where each step adds a small amount of Gaussian noise *(Lecture 26)*.

$$p(x_t|x_{t-1}) = \mathcal{N}(x_t; \sqrt{1-\beta_t}x_{t-1}, \beta_t I)$$

The sequence of variances $\{\beta_t\}_{t=1}^T$ is called the "noise schedule." A key property is that we can sample x_t directly from x_0 in a closed form, which is crucial for efficient training *(HW 13, Problem 4a)*. Let $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{i=1}^t \alpha_i$. Then:

$$q(x_t|x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}x_0, (1-\bar{\alpha}_t)I)$$

Conceptual Example: Binary Diffusion *(Discussion 13, Problem 1)* To build intuition, consider a diffusion process for binary images where pixels are $\{+1, -1\}$. At each step, for each pixel, we flip its value with a small probability δ. As the number of steps $t \to \infty$, the probability of any given pixel having been replaced by a fresh random coin flip approaches 1. Therefore, the final image Y_T becomes a collection of i.i.d. fair coin flips, analogous to pure noise.

3.3 The Reverse Process

The goal is to learn the reverse transitions $p(x_{t-1}|x_t)$. A key theoretical result is that if the forward step noise is small and Gaussian, the reverse conditional is also approximately Gaussian *(Lectures 26, 27)*.

$$p(x_{t-1}|x_t) \approx \mathcal{N}(x_{t-1}; \vec{\mu}_\theta(x_t, t), \sigma_t^2 I)$$

The variance σ_t^2 can be fixed based on the noise schedule. The main task is to train a neural network (often a U-Net) to predict the mean $\vec{\mu}_\theta(x_t, t)$ given the noisy image x_t and the timestep t *(Lecture 26)*.

3.3.1 Training the Denoising Model

The model is trained to predict the mean of the reverse step. As shown in the continuous-time limit, this mean is related to the **score function** $\nabla_{x_t} \log p_t(x_t)$ *(Lecture 27)*.

$$\mu(x_t) = x_t + \sigma_t^2 \Delta t \nabla_{x_t} \log p_t(x_t)$$

In practice, this is equivalent to predicting the noise ϵ that was added to x_0 to get x_t, or predicting the original clean image x_0 itself *(Lecture 26)*. Training proceeds as follows *(Lecture 26)*:

1. Sample a clean image x_0 from the dataset.
2. Sample a random timestep $t \in \{1, \ldots, T\}$.
3. Sample noise $\epsilon \sim \mathcal{N}(0, I)$ and create the noisy image $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$.
4. Train the network $\epsilon_\theta(x_t, t)$ to predict the noise ϵ using a simple loss, like Mean Squared Error: $\|\epsilon_\theta(x_t, t) - \epsilon\|^2$.

3.4 Generation: DDPM vs. DDIM

Once trained, the model can generate new samples by starting with noise and running the reverse process. There are two main approaches.

3.4.1 Stochastic Sampling (DDPM)

Denoising Diffusion Probabilistic Models (DDPM) simulate the stochastic reverse process *(Lecture 27)*.

1. Start with pure noise: $x_T \sim \mathcal{N}(0, I)$.
2. For $t = T, \ldots, 1$:
   - Predict the mean $\mu_\theta(x_t, t)$ using the trained network.
   - Sample the next step: $x_{t-1} \sim \mathcal{N}(\mu_\theta(x_t, t), \sigma_t^2 I)$.
3. The final result x_0 is the generated sample.

3.4.2 Deterministic Sampling (DDIM)

Denoising Diffusion Implicit Models (DDIM) observe that we only need to match the marginal distributions $p_t(x_t)$, not the full joint distribution of the reverse path. This allows for a deterministic path from noise to data *(Lecture 27)*. The DDIM update rule can be viewed as taking a step in the direction of the predicted clean image x_0:

$$\hat{x}_{t-\Delta t} = \sqrt{\bar{\alpha}_{t-\Delta t}}\hat{x}_{0|t} + \sqrt{1-\bar{\alpha}_{t-\Delta t}} \cdot \frac{x_t - \sqrt{\bar{\alpha}_t}\hat{x}_{0|t}}{\sqrt{1-\bar{\alpha}_t}}$$

where $\hat{x}_{0|t}$ is the model's prediction of x_0 given x_t. This process is deterministic and often allows for faster sampling with fewer steps than DDPM *(Lecture 27)*. From a theoretical perspective, DDIM learns a velocity field for an Ordinary Differential Equation (ODE) that flows points from the noise distribution to the data manifold *(Lecture 27)*.

Practice Problem: DDPM/DDIM Math *(HW 13, Problem 1)* This problem explores the DDPM and DDIM updates in detail under simplifying assumptions, asking to derive variances and update rules. For example, it shows that the conditional mean of $x_{t-\Delta t}$ given x_t for a process starting from a Gaussian is $\mathbb{E}[x_{t-\Delta t}|x_t] = \frac{\sigma^2(t-\Delta t)}{\sigma^2 t}x_t$. This is the ideal denoiser in this simple case, and both DDPM and DDIM steps are designed to approximate this behavior.

4 Post-Training of Large Language Models (LLMs)

After pre-training on a vast text corpus, LLMs undergo several stages of post-training to align their behavior with human instructions and preferences *(Lecture 26)*.

4.1 Supervised Fine-Tuning (SFT)

SFT is the first step, where the pre-trained model is fine-tuned on a smaller, high-quality dataset of instruction-response pairs *(Lectures 24, 25)*.

- **Masked Loss**: The standard cross-entropy loss is applied, but only on the tokens of the response. The loss for the prompt tokens is masked to zero. This teaches the model the format of answering questions without unlearning its general language model capabilities *(Lecture 25)*.
- **Chain-of-Thought (CoT)**: A key discovery was that prompting models to "think step by step" improves performance on reasoning tasks. SFT datasets now often include these intermediate reasoning steps to train the model to produce more deliberative outputs *(Lectures 24, 25)*.

4.1.1 Challenge: Catastrophic Forgetting

A major challenge in fine-tuning is **catastrophic forgetting**: the tendency of a model to lose performance on previously learned tasks when trained on a new one *(Discussion 12, Problem 2)*. This happens when the parameter updates for the new task move the model's weights far away from the optimal region for the old tasks.

Practice Problem: Fine-Tuning Strategies *(Discussion 12, Problem 2b)* This problem compares three strategies for learning a new task while retaining old knowledge:

1. **Frozen Feature Extraction**: Freeze the base model and train a new head for the new task. Good for old task performance, but can be suboptimal for the new task.

2. **Full Fine-Tuning**: Train the entire model on the new task. Best performance on the new task in most cases, but high risk of catastrophic forgetting (bad for old task performance).

3. **Joint Training**: Train on a mix of old and new data. Good for both, but requires storing and re-using old data, which can be infeasible.

A modern solution to this problem is **Parameter-Efficient Fine-Tuning (PEFT)**, such as Low-Rank Adaptation (LoRA). LoRA freezes the pre-trained weights and injects small, trainable low-rank matrices into the model. This significantly reduces the number of trainable parameters, mitigating forgetting and reducing computational cost *(HW 11, Problem 1)*.

5 Improving LLM Performance via Inference

Beyond training, performance can be improved by using more computation at inference time *(Lectures 24, 25)*.

5.1 Basic Test-Time Compute Techniques

- **Prompt Engineering**: Modifying the prompt to elicit better behavior, e.g., adding "Think step by step" *(Lecture 25)*.
- **Best-of-N Sampling**: Generate N different responses to a prompt (using non-zero temperature) and select the best one using a criterion like majority vote, highest sequence probability, or an external judge model *(Lectures 24, 25)*.

5.2 Advanced Sampling: Reasoning with Sampling

This approach posits that high-quality reasoning sequences often have higher probability under the base model. Therefore, we can improve performance by sampling from a "sharpened" distribution that upweights these high-probability sequences *(Lecture 25)*. We sample from a modified distribution:

$$p_{\text{desired}}(\vec{x}) \propto [p_{\text{base}}(\vec{x})]^{\alpha} \quad \text{for } \alpha > 1$$

This is fundamentally different from low-temperature sampling, as it considers the probability of the entire sequence, not just the next token, requiring non-trivial sampling methods like MCMC *(Lecture 25)*. The **Power Sampling** algorithm uses MCMC to generate samples from this intractable distribution, showing performance competitive with models that have undergone extensive RL fine-tuning *(Lecture 25)*.

6 Improving LLM Performance via Reinforcement Learning

RL provides a powerful framework for fine-tuning LLMs based on complex, non-differentiable rewards, such as human preferences or task success.

6.1 RL with Verifiable Rewards (RLVR)

When an automated "autograder" exists for a task (e.g., checking a math answer or running unit tests), we can use RLVR *(Lectures 25, 26)*. The ScaleRL paper proposes a robust RL objective

(Lectures 25, 26):

$$J_{\text{ScaleRL}}(\theta) = \mathbb{E}_{x\sim D,\{y_i\}\sim\pi_{\text{gen}}}\left[\frac{1}{|G|}\frac{1}{\sum|y_g|}\sum_{i=1}^{G}\sum_{t=1}^{|y_i|}\text{sg}\left(\min(\rho_{i,t},\epsilon)\right)\hat{A}_i^{\text{norm}}\log\pi^{\theta}_{\text{train}}(y_{i,t}|y_{i,<t})\right]$$

Key components include:

- **REINFORCE Structure**: The loss has the form $\mathbb{E}[R\cdot\nabla\log\pi]$, where the reward signal is the normalized advantage $\hat{A}_i^{\text{norm}}$ *(Lecture 25)*.
- **Normalized Advantage**: $\hat{A}_i^{\text{norm}} = (r_i - \bar{r})/\sigma_r$. Subtracting the batch mean reward $\bar{r}$ provides a baseline, rewarding generations better than average and penalizing those worse. Normalizing by the standard deviation σ_r stabilizes training *(Lecture 25)*.
- **Importance Sampling**: The ratio $\rho_{i,t} = \pi_{\text{train}}/\pi_{\text{gen}}$ corrects for the mismatch between the policy being trained and the (older) policy used to generate the data *(Lecture 25)*.
- **Clipping and Stop Gradient**: The 'sg' (stop gradient) on $\rho_{i,t}$ prevents incorrect gradients. Clipping prevents the importance weights from vanishing or exploding, improving stability *(Lectures 25, 26)*.

6.2 Reinforcement Learning from Human Feedback (RLHF)

When rewards are not automatically verifiable, we use human feedback. The standard RLHF pipeline has three stages *(Lecture 26, HW 13, Problem 2)*:

1. **SFT**: Fine-tune a base model on instruction-response data.
2. **Reward Modeling**: Collect human preference data (e.g., which of two responses is better). Train a separate reward model (RM) to predict which response a human would prefer.
3. **RL Fine-Tuning**: Use the trained RM as the reward function to fine-tune the SFT model using an RL algorithm like PPO.

During RL fine-tuning, a KL-divergence penalty is added to prevent the policy from deviating too far from the original SFT model, mitigating catastrophic forgetting *(Lecture 26)*. The objective is:

$$\max_{\pi}\mathbb{E}_{y\sim\pi(\cdot|x)}[r(x,y)] - \beta D_{\text{KL}}[\pi\|\pi_{\text{ref}}]$$

The optimal policy for this objective has a specific form *(Lecture 26, HW 13, Problem 2b)*:

$$\pi^*(y|x) = \frac{1}{Z(x)}\pi_{\text{ref}}(y|x)\exp\left(\frac{1}{\beta}r(x,y)\right)$$

This shows that the optimal policy re-weights the reference policy's distribution to favor outputs with higher rewards.

6.3 Direct Preference Optimization (DPO)

DPO is a more recent method that bypasses the need for an explicit reward model, learning directly from preference data *(Lecture 26)*. Given preference triplets (x, y_w, y_l), where y_w is the preferred (winner) response and y_l is the dispreferred (loser) response, DPO optimizes the following loss *(Lecture 26, HW 13, Problem 2e)*:

$$\mathcal{L}_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(x,y_w,y_l)\sim D}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\right)\right]$$

where σ is the sigmoid function. This loss is derived by substituting the optimal policy form from the KL-regularized RL objective into the Bradley-Terry model for pairwise preferences and formulating a maximum likelihood objective *(HW 13, Problem 2c-e)*. The term inside the sigmoid is the difference in rewards for the winner and loser under an *implicit* reward model defined by the policy change: $\hat{r}(x, y) = \beta \log(\pi_\theta(y|x)/\pi_{\text{ref}}(y|x))$ *(Lecture 26)*. This provides a stable, direct way to optimize for human preferences.

7 General Techniques and Related Concepts

7.1 Generalizable Ideas from VAEs

The development of VAEs introduced several powerful, broadly applicable techniques *(Lecture 24)*:

1. **KL Divergence as a Regularizer**: A general method to encourage an internal distribution of a network to match a target prior.
2. **The Reparameterization Trick**: A fundamental technique for backpropagating through stochastic nodes in a computation graph.
3. **Stochasticity in SGD**: Treating model-induced randomness (like sampling z) as another source of stochasticity, similar to mini-batch sampling, which SGD can handle.

7.2 Application: Quantization-Aware Training (QAT)

Quantization reduces the numerical precision of a model's weights (e.g., from float32 to int8) to save memory and speed up inference. Naively quantizing a trained model can hurt performance *(Lecture 24)*.

QAT trains the model to be robust to quantization from the start. This is done using a **straight-through estimator** *(Lecture 24)*.

- **Forward Pass**: Weights are quantized (a non-differentiable rounding operation) before use.
- **Backward Pass**: During backpropagation, the gradient is passed through the quantization step as if it were the identity function.

This allows the model to learn weights that are naturally close to the quantization levels, preserving performance *(Lecture 24)*. This works because neural networks are inherently robust to small amounts of noise, and the quantization error can be seen as structured noise *(HW 11, Problem 7b)*.